Reviewer Pack — Minimal Diophantine Exponent and Communication Complexity Ra…

Entry b9f19817991d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 01:07:46 UTC

Minimal Diophantine Exponent and Communication Complexity Rank Variance Bound

Entry ID: `b9f19817991d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 01:07:46 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Equations) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any given n-ary Boolean function, the minimal diophantine exponent for its associated polynomial is linearly correlated with its communication complexity rank variance, such that μ(π) = Θ(w(φ_π)), where μ(π) is the minimal diophantine exponent of π, φ_π is the associated Tseitin formula, and w(φ_π) is the communication complexity rank.

Rationale (proposer's reasoning):

> Diophantine equations are known to be related to Boolean functions through their corresponding polynomials. By examining the minimal diophantine exponent, we may uncover structural properties of the Boolean function that influence its communication complexity rank. This bridge could expose novel relationships between number theory and complexity theory.

Taxonomy category: `DiophantineNumberTheoryCommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `18992eb7116b80c4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between the minimal diophantine exponent and the communication complexity rank variance exceeds 0.7 for at least 80% of the seeds, with no seed's correlation being below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Diophantine Exponent AND Communication Complexity Rank Variance Bound - Diophantine Equations IN NUMBER THEORY AND Matrix Rank IN Communication Complexity - Tseitin formula AND μ(π) = θ(w(φ_π))

Top relevant hits considered: - [http://arxiv.org/abs/1609.06986v1] More on Diophantine sextuples - [http://arxiv.org/abs/1709.09951v2] Asymptotically tight worst case complexity bounds for initial-value problems with nonadaptive information - [http://arxiv.org/abs/quant-ph/0405018v2] Improved Bounds on the Randomized and Quantum Complexity of Initial-Value Problems - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/2303.12927v2] Study of the $f_0(980)$ and $f_0(500)$ Scalar Mesons through the Decay $D_s^+ \to \pi^+\pi^- e^+\nu_e$ - [http://arxiv.org/abs/2411.07730v2] Study of the light scalar $a_0(980)$ through the decay $D^0 \to a_0(980)^- e^+\nu_e$ with $a_{0}(980)^- →ηπ^- - [http://arxiv.org/abs/1603.09653v2] Observation of an anomalous line shape of the $\eta'\pi^+\pi^-$ mass spectrum near the $p\bar{p}$ mass threshold in

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.5s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tseitin_formula(boolean_func, n):
        # Simplified Tseitin formula generation
        clauses = []
        for i in range(2**n):
            clause = []
            for j in range(n):
                if boolean_func[i] & (1 << j):
                    clause.append(j)
```

```
            else:
                clause.append(-j - 1)
        clauses.append(clause)
    return clauses

def diophantine_exponent(clauses):
    # Simplified calculation of minimal diophantine exponent
    return len(clauses) / n

def communication_complexity_rank_variance(clauses):
    # Simplified calculation of communication complexity rank variance
    rank = len(clauses)
    return (rank - 1) ** 2

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    boolean_func = generate_boolean_function(n)
    clauses = tseitin_formula(boolean_func, n)
    diophantine_exp = diophantine_exponent(clauses)
    rank_variance = communication_complexity_rank_variance(clauses)

    results.append({
        "n": n,
        "diophantine_exponent": diophantine_exp,
        "rank_variance": rank_variance
    })

if not results:
    return {
        "metric_name": "diophantine_exponent",
        "metric_value": 0.0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

diophantine_values = [r["diophantine_exponent"] for r in results]
rank_variance_values = [r["rank_variance"] for r in results]

mean_diophantine = sum(diophantine_values) / len(diophantine_values)
mean_rank_variance = sum(rank_variance_values) / len(rank_variance_values)

correlation_coefficient = (sum((d - mean_diophantine) * (v - mean_rank_variance) for d, v in z
                           math.sqrt(sum((d - mean_diophantine)**2 for d in diophantine_values
                                     sum((v - mean_rank_variance)**2 for v in rank_variance_va

return {
    "metric_name": "diophantine_exponent",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(r["n"] for r in results),
    "conjecture_holds": correlation_coefficient >= 0.7,
    "counterexample": "" if correlation_coefficient >= 0.7 else f"correlation_coefficient={cor
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        result = f"FALSIFIED counterexample=\"correlation_coefficient_below_0.7\" first_failing_se

    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete the analysis required to evaluate the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the allocated time.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12961
2	preregistration	ollama_remote	glm4:latest	0	0	10055
3	novelty	ollama_remote	glm4:latest	0	0	8491

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	9766
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20158
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21851
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8716
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10844
9	judge	ollama_remote	glm4:latest	0	0	25348

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 128190 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b9f19817991d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b9f19817991d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b9f19817991d.tar.gz` (if generated)